



University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

Was kostet Observability?

Implementierung einer Microservice-Anwendung mit .NET und Angular
am Beispiel eines Projektmanagement-Werkzeugs und quantitative
Analyse des Overheads von OpenTelemetry

Exposé

Projektarbeit im Master Studiengang Informatik
Wintersemester 2026/27

Autor: Erlind Sejdiu
Matrikelnummer: 106357

Korrektor: [Name der betreuenden Person]

Inhaltsverzeichnis

1	Motivation	1
2	Problemstellung	1
3	Stand der Technik	1
4	Ziele	2
5	Vorgehen	2
5.1	Teil 1: Projektmanagement-Werkzeug und Instrumentierung	2
5.2	Teil 2: Messung	3
6	Zeitplan	4
	Literaturverzeichnis	4

1 Motivation

In Microservice-Architekturen durchläuft eine einzelne Nutzeraktion mehrere Dienste, Datenbanken und Nachrichtenkanäle [1]. Tritt ein Fehler auf oder wird eine Anfrage langsam, ist die Ursache oft nicht in einem einzelnen Dienst zu finden, sondern im Zusammenspiel mehrerer Komponenten. *Observability* macht den inneren Zustand eines solchen Systems anhand von Traces, Metriken und Logs nachvollziehbar [2]. OpenTelemetry hat sich dafür als herstellerunabhängiger De-facto-Standard etabliert und bietet einheitliche Schnittstellen für viele Sprachen und Plattformen [3], [4].

Observability ist mit Kosten verbunden. Jeder Span, jede Metrik und jeder Log-Eintrag kostet Rechenzeit und Arbeitsspeicher, erzeugt Netzwerkverkehr und belegt Speicherplatz. Zusätzlich kann die Instrumentierung die Antwortzeiten verlängern, die Nutzerinnen und Nutzer direkt wahrnehmen. Wie umfangreich instrumentiert und wie stark gesampelt wird, wird in der Praxis dennoch nach Erfahrung oder den Voreinstellungen der eingesetzten Bibliotheken entschieden. Qualitative Empfehlungen gibt es viele, belastbare quantitative Aussagen darüber, wie groß der Overhead in einer realistischen Anwendung ist und wovon er abhängt, dagegen kaum.

Die Arbeit ist daher zweigeteilt. Im ersten Teil wird ein Projektmanagement-Werkzeug als Microservice-Anwendung mit .NET und Angular entwickelt und mit OpenTelemetry instrumentiert. Im zweiten Teil wird an dieser Anwendung der Einfluss von OpenTelemetry auf Performanz und Speicherbedarf quantitativ gemessen.

2 Problemstellung

Nach ISO/IEC 25010 verbessert Observability die *Analysierbarkeit* eines Systems, belastet aber die *Leistungseffizienz* mit ihren Teilmerkmalen *Zeitverhalten* und *Ressourcennutzung* [5]. Um beide Qualitätsmerkmale gegeneinander abwägen zu können, muss der Preis der Observability bekannt sein. Offen ist derzeit:

1. um wie viel sich Antwortzeiten, Durchsatz einer .NET-Microservice-Anwendung durch OpenTelemetry verschlechtern,
2. wie viel zusätzlichen Arbeitsspeicher die Dienste benötigen und welches Datenvolumen die Telemetriedaten verursachen,
3. wie stark Konfigurationsparameter wie die erfassten Signale, die Granularität, das Sampling und die Art des Exports den Overhead beeinflussen.

Eine Antwort auf diese Fragen lässt sich nicht an einem minimalen Beispiel gewinnen. Der Overhead hängt davon ab, wie viele Dienste an einer Anfrage beteiligt sind, wie viel Arbeit jeder Dienst im Verhältnis zur Instrumentierung leistet und wie viele Datenbank- und Netzwerkaufrufe dabei entstehen. Benötigt wird daher eine Anwendung, die typische Anfragen eines realen Systems abbildet und dennoch kontrolliert und wiederholbar gemessen werden kann.

Abgrenzung. Die Anwendung dient als realistisches Messobjekt und ist kein produktionsreifes Produkt. Nicht betrachtet werden der Vergleich verschiedener Observability-Backends (Granfa oder Datadog). Die Analyse zwischen unterschiedlichen Observability-Backends würde den Rahmen dieser Projektarbeit sprengen.

3 Stand der Technik

//

4 Ziele

- 1: Wie stark verändern sich Antwortzeit, maximaler Durchsatz und CPU-Auslastung durch OpenTelemetry?
- 2: Wie viel zusätzlichen Arbeitsspeicher benötigen die Dienste, und welches Datenvolumen verursachen die Telemetriedaten?
- 3: Welchen Einfluss haben Signale, Granularität, Sampling-Rate und Exportverfahren auf FF1 und FF2?
- 4: Wie wirkt sich die Instrumentierung des Angular-Frontends auf Bundle-Größe und Ladezeit aus?

Ergebnis von Teil 1 ist eine lauffähige, in Containern bereitstellbare Anwendung, deren OpenTelemetry-Konfiguration sich ohne Codeänderung umschalten lässt – bis hin zum Betrieb ganz ohne OpenTelemetry. Ergebnis von Teil 2 sind eine reproduzierbare Messumgebung, die Beantwortung von FF1–FF4 sowie daraus abgeleitete Konfigurationsempfehlungen für Entwicklungsteams.

5 Vorgehen

5.1 Teil 1: Projektmanagement-Werkzeug und Instrumentierung

Mehrere Personen planen gemeinsam Projekte: Tasks werden angelegt, mit Abhängigkeiten verknüpft und in einer Kanban-, Wochen- und Monatsansicht dargestellt. Beim Verknüpfen wird geprüft, dass keine zyklischen Abhängigkeiten entstehen. Projekte werden über Rollen mit anderen geteilt; Änderungen erscheinen bei allen Beteiligten in Echtzeit.

Abbildung 1 zeigt die geplante Architektur. Das Angular-Frontend kommuniziert ausschließlich mit einem API-Gateway, das Anfragen authentifiziert und an die zuständigen Dienste weiterleitet. Jeder Dienst verwaltet seine Daten in einer eigenen Datenbank. Zur Berechtigungsprüfung rufen Projekt- und Task-Dienst den Nutzer- und Team-Dienst synchron auf. Änderungen werden zusätzlich als Ereignisse über einen Nachrichtenbroker veröffentlicht. Diese Ereignisse verarbeitet der Benachrichtigungsdienst asynchron und sendet daraus Echtzeit-Updates an das Frontend.

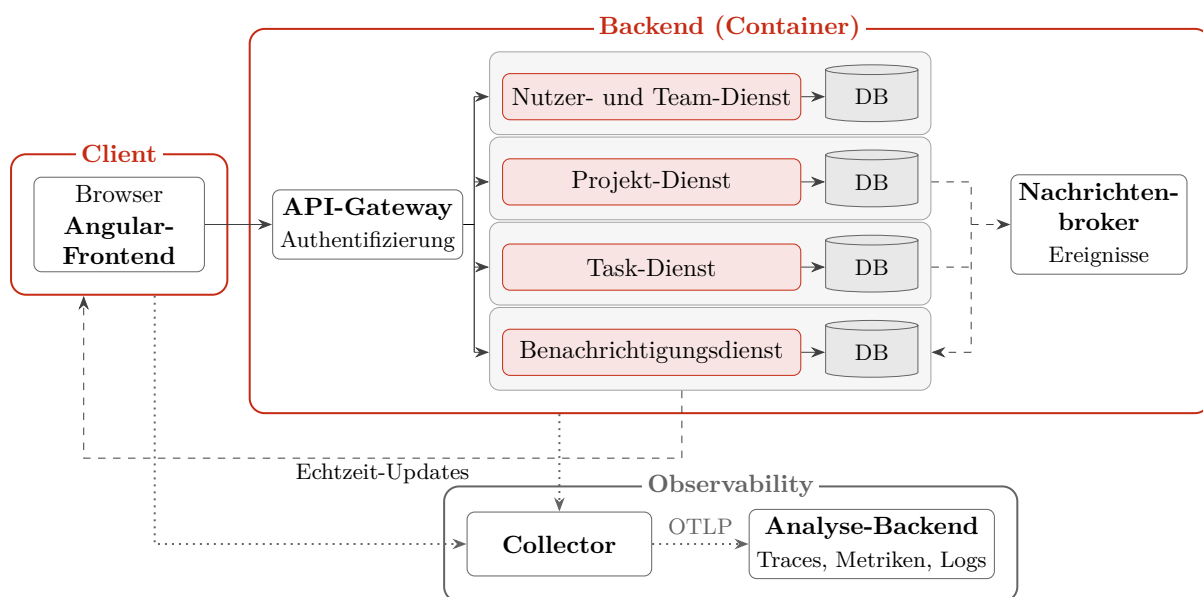


Abbildung 1: Geplante Architektur des Projektmanagement-Werkzeugs (vereinfacht).
Durchgezogene Pfeile: synchrone Aufrufe; gestrichelt: Ereignisse; gepunktet: Telemetriedaten.

Die Domäne enthält damit unterschiedliche Anfragepfade, die für die Messung relevant sind: große Lesezugriffe (Kalenderansichten, Board), kurze Schreibzugriffe (Task verschieben), rechenintensive Logik (Zyklusprüfung), synchrone Aufrufketten über mehrere Dienste und asynchrone Verarbeitung.

Instrumentiert wird zunächst automatisch, sodass eingehende und ausgehende Anfragen, Datenbankzugriffe und Nachrichten ohne eigenen Code erfasst werden. Ergänzt wird dies um manuelle Spans und Metriken für fachliche Vorgänge wie die Zyklusprüfung. Das Frontend gibt den Trace-Kontext an das Backend weiter, sodass eine Nutzeraktion vom Klick bis zur Datenbank als ein Trace sichtbar wird. Alle Dienste senden ihre Daten an einen gemeinsamen Collector, der sie an ein Analyse-Backend weiterleitet. Welche Signale erfasst werden, wie fein instrumentiert und wie stark gesampelt wird, lässt sich über Konfiguration umschalten. Wichtige Entwurfsentscheidungen werden als Architecture Decision Records dokumentiert [6].

5.2 Teil 2: Messung

Versuchsdesign. Gemessen wird gegen eine Basislinie ohne OpenTelemetry. Ausgehend von einer Standardkonfiguration wird jeweils ein Faktor variiert, um seinen Einfluss isoliert betrachten zu können:

Faktor	Stufen
Instrumentierung	keine; SDK ohne Export; automatisch; zusätzlich manuell (grob/fein)
Signale	Traces; zusätzlich Metriken; zusätzlich Logs
Sampling	0 %, 1 %, 10 %, 100 %
Export	verschiedene Übertragungsprotokolle; gebündelter vs. sofortiger Export
Last	25 %, 50 %, 75 % des maximalen Durchsatzes der Basislinie

Messgrößen. Erfasst werden Antwortzeiten und Durchsatz aus Sicht eines Lastgenerators, CPU und Arbeitsspeicher je Dienst einschließlich Speicherallokationen und Garbage Collection, das Datenvolumen zum Collector und der belegte Speicherplatz im Backend. Im Frontend werden Bundle-Größe und Ladezeit anhand der Core Web Vitals bewertet [7].

Messumgebung. Der Lastgenerator führt gewichtete Szenarien aus, die typische Nutzung abbilden: Board laden, Wochen- und Monatsansicht öffnen, Tasks anlegen, verschieben und verknüpfen. Grundlage ist ein fester, generierter Datenbestand, damit alle Konfigurationen unter gleichen Bedingungen gemessen werden. Ressourcen werden mit Werkzeugen erfasst, die unabhängig von OpenTelemetry arbeiten, damit die Messung das Ergebnis nicht verfälscht. Einzelne Operationen werden zusätzlich isoliert per Micro-Benchmark gemessen. Alle Container erhalten feste Ressourcenlimits, jede Konfiguration wird nach einer Aufwärmphase mehrfach und automatisiert gemessen. [\[Messrechner klären.\]](#)

Auswertung. Angegeben werden Perzentile mit Konfidenzintervallen und der relative Overhead zur Basislinie; Unterschiede werden mit dem Mann-Whitney-U-Test auf Signifikanz geprüft [8], [9]. Die Ergebnisse werden mit bestehenden Studien verglichen [10]. Abschließend werden die Übertragbarkeit auf reale Systeme sowie der Einfluss von Messumgebung und Softwareversionen diskutiert.

6 Zeitplan

Phase	Ziel	Artefakt
1 Einarbeitung	OpenTelemetry, Messmethodik	Literaturübersicht
2 Implementierung	Dienste und Angular-Frontend	Lauffähige Anwendung, ADRs
3 Instrumentierung	Backend, Frontend, Collector	Umschaltbare Konfiguration
4 Messumgebung	Lastszenarien, Automatisierung, Pilotmessungen	Mess- und Auswertungsskripte
5 Messungen	Alle Konfigurationen messen	Rohdaten
6 Auswertung	Statistik, Empfehlungen	Diagramme, Tabellen
7 Ausarbeitung	Schriftliche Dokumentation	Ausarbeitung, Code und Daten im Repository

Literaturverzeichnis

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2. Aufl. Sebastopol, CA: O'Reilly Media, 2021.
- [2] C. Majors, L. Fong-Jones und G. Miranda, *Observability Engineering: Achieving Production Excellence*. Sebastopol, CA: O'Reilly Media, 2022.
- [3] Cloud Native Computing Foundation. „OpenTelemetry – Project Page,“ besucht am 28. Sep. 2026. Adresse: <https://www.cncf.io/projects/opentelemetry/>
- [4] OpenTelemetry Authors. „OpenTelemetry Documentation,“ besucht am 28. Sep. 2026. Adresse: <https://opentelemetry.io/docs/>
- [5] *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, 2023.
- [6] M. Nygard. „Documenting Architecture Decisions,“ besucht am 28. Sep. 2026. Adresse: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [7] Google. „Web Vitals,“ besucht am 29. Sep. 2026. Adresse: <https://web.dev/articles/vitals>
- [8] A. Georges, D. Buytaert und L. Eeckhout, „Statistically Rigorous Java Performance Evaluation,“ in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 2007.
- [9] T. Kalibera und R. Jones, „Rigorous Benchmarking in Reasonable Time,“ in *Proceedings of the 2013 International Symposium on Memory Management (ISMM '13)*, 2013.
- [10] D. G. Reichelt, S. Kühne und W. Hasselbring, „Towards Solving the Challenge of Minimal Overhead Monitoring,“ in *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23 Companion)*, 2023.